

GUÍA MAESTRA DE DEMOSTRACIÓN Y DEFENSA ORAL (LG14)

Asignatura: Tecnologías Emergentes / Big Data | Proyecto: Clúster Apache Hadoop en Docker

Grupo 4: Joel Saavedra | Mauricio Linaja | Rommel Gutierrez

Criterio Rector de Evaluación LG14: Encontrar → Comprender → Desplegar → Probar → Analizar

1. Cronograma de la Presentación (7 a 10 Minutos)

Fase	Tiempo	Objetivo Principal de la Demostración
1. Introducción y Selección	1 min	Justificar la elección de Apache Hadoop (HDFS/MapReduce) y registro formal del repositorio.
2. Arquitectura (4 Capas)	2 min	Explicar las 4 capas: Host, HDFS (NameNode/DataNode), YARN (RM/NM/History) y Persistencia.
3. Demostración en Vivo	4 min	Validar docker ps, Web UI HDFS (localhost:9870), comandos HDFS y Job MapReduce en YARN.
4. Comparativa con docker-hadoop	2 min	Defender la tabla de 11 criterios: Hadoop Streaming y agilidad vs stack tradicional.
5. Conclusiones y Cierre	1 min	Presentar historial de commits atómicos y responder preguntas defensivas.

2. Guion de Oratoria y Acciones en Pantalla

FASE 1: Introducción y Selección del Repositorio (1 Minuto)

Acción en Pantalla: Mostrar la portada del documento README.md (Secciones 1 y 2).

"Buenas tardes docente y compañeros. Para la práctica LG14 seleccionamos el repositorio público **hadoop-hdfs-map-reduce-docker** de Martin Castro Alvarez. Elegimos Apache Hadoop versión 3.2.1 porque implementa el estándar industrial de Big Data para almacenamiento distribuido en HDFS y procesamiento paralelo MapReduce vía Hadoop Streaming, cumpliendo con la consigna de ser un entorno completamente contenerizado y reproducible mediante Docker Compose."

FASE 2: Exposición de la Arquitectura en 4 Capas (2 Minutos)

Acción en Pantalla: Mostrar el Diagrama Mermaid de Arquitectura en el README.md (Sección 4).

1. Capa Superior (Host Local Windows): "Desde aquí interactuamos con el clúster a nivel de consola mediante Docker CLI ejecutando docker exec para ingresar al NameNode, y a nivel visual desde el Navegador Web en los puertos clave: 9870 (HDFS), 8088 (YARN) y 8188 (HistoryServer)."

2. Red y Capa HDFS (Almacenamiento): "Los contenedores conviven en la red Bridge privada hadoop-network. El NameNode administra el namespace e inodos en memoria y atiende en el puerto 9000; el DataNode guarda físicamente los bloques de datos y reporta heartbeats y métricas en el puerto 9864."

3. Capa YARN (Cómputo Distribuido): "El ResourceManager coordina recursos globales, el NodeManager ejecuta los contenedores de cómputo Mapper/Reducer y el HistoryServer almacena logs de trabajos finalizados."

4. Capa de Persistencia (Inferior): "Volúmenes nombrados hadoop_namenode, hadoop_datanode y hadoop_historyserver preservan metadatos, bloques y logs en el disco del host ante reinicios."

Cierre de Arquitectura: "Gracias a esta arquitectura, cuando ejecutamos un hdfs dfs -put, el cliente le pide ubicación al NameNode, el archivo se transfiere y almacena en bloques dentro del DataNode, persiste en el volumen y lo audítamos en tiempo real en el puerto 9870."

FASE 3: Despliegue y Demostración Práctica en Vivo (4 Minutos)**Paso 1: Estado de los Contenedores en Terminal:**

`docker ps` → Confirmamos 5 contenedores en estado Up y puertos activos.

Paso 2: Ingreso a la Consola Nativa del NameNode:

`winty docker exec -it namenode bash` (Acceso directo a Linux en `root@namenode:/#`).

Paso 3: Verificación de Salud del Clúster HDFS:

`hdfs dfsadmin -report` → Reporta Live datanodes: 1, capacidad de ~940 GB y 0 bloques corruptos.

Paso 4: Creación de Directorio en HDFS (Requisito Obligatorio 1):

`hdfs dfs -mkdir -p /user/laboratorio`
`hdfs dfs -ls /user` → Crea la ruta distribuida en el namespace de inodos.

Paso 5: Carga de Archivo con Integrantes (Requisito Obligatorio 2):

`echo "Sistemas Distribuidos Big Data - Saavedra, Linaja, Gutierrez (Grupo 4)" > /tmp/prueba_hdfs.txt`
`hdfs dfs -put -f /tmp/prueba_hdfs.txt /user/laboratorio/`

Paso 6: Consulta por Consola y Web UI (Requisito Obligatorio 3):

`hdfs dfs -ls /user/laboratorio`

`hdfs dfs -cat /user/laboratorio/prueba_hdfs.txt`

Navegación Web UI: Ir a <http://localhost:9870> > Utilities > Browse the file system > /user/laboratorio/prueba_hdfs.txt. Clic en Head the file (first 32K) para ver el contenido y auditar el Block ID.

Paso 7: Procesamiento Distribuido MapReduce con Hadoop Streaming:

`hdfs dfs -mkdir -p /input_mr && echo -e "hadoop bigdata distributed hdfs\nhadoop mapreduce bigdata\nhdfs cluster saavedra linaja gutierrez grupo4" | hdfs dfs -put -f - /input_mr/data.txt`
`hdfs dfs -rm -r -f /output_mr`
`hadoop jar /opt/hadoop-3.2.1/share/hadoop/tools/lib/hadoop-streaming-3.2.1.jar -files /app/mapper.sh,/app/reducer.sh -input /input_mr/data.txt -output /output_mr -mapper mapper.sh -reducer reducer.sh`
`hdfs dfs -cat /output_mr/part-00000`

FASE 4: Análisis Comparativo Completo con docker-hadoop (2 Minutos)

Acción en Pantalla: Mostrar la Tabla Comparativa de 11 Criterios (Sección 8 del README.md).

"En comparación con docker-hadoop de Big Data Europe, nuestro proyecto destaca por su despliegue autocontenido sin requerir configuraciones de entorno externas complejas, facilitando el procesamiento analítico inmediato mediante scripts de Hadoop Streaming sin necesidad de compilar pesados archivos JAR en Java."

FASE 5: Conclusiones y Cumplimiento de Entrega (1 Minuto)

Acción en Pantalla: Ejecutar `git log --oneline` en terminal.

"Finalizamos demostrando el historial de commits atómicos realizados en nuestro repositorio **bigdata-grupo-4**, cumpliendo con la totalidad de requisitos del documento LG14. Quedamos a disposición para preguntas."

3. Tabla Comparativa Exhaustiva (11 Criterios Oficiales)

Criterio Oficial	docker-hadoop (Referencia BDE)	hadoop-hdfs-map-reduce-docker (Nuestro Proyecto)
1. Tecnología Principal	Apache Hadoop (HDFS + YARN)	Apache Hadoop (HDFS + MapReduce Streaming)
2. Docker	Sí (Imágenes Debian GNU/Linux)	Sí (Imágenes Debian Buster OpenJDK 8)
3. Docker Compose	Sí (Formato v2/v3 con .env externo)	Sí (Orquestación modular autocontenida)
4. Contenedores	5 contenedores monolíticos	5 contenedores especializados (NameNode, DataNode, YARN, etc.)
5. Almacenamiento	HDFS multi-nodo tradicional	HDFS modularizado con persistencia nombrada
6. Procesamiento	MapReduce sobre Java clásico	MapReduce ágil vía Hadoop Streaming (Bash/Python)
7. Interfaces Web	NameNode (9870), YARN (8088), History (8188)	NameNode Web UI (9870), YARN (8088), History (8188)
8. Persistencia	Volúmenes montados en carpetas de contenedor	Volúmenes nombrados dedicados (hadoop_namenode, hadoop_datanode)
9. Complejidad Inst.	Media-Alta (dependencia estricta de archivo .env)	Muy Baja (despliegue directo con docker compose up -d)
10. Documentación	Genérica orientada al stack BDE	Práctica y enfocada en pipelines MapReduce y HDFS
11. Caso de Uso	Infraestructura base para conectar Hive/Spark	Laboratorio ágil de almacenamiento masivo y algoritmos paralelos

4. Banco de Preguntas Defensivas Resueltas

Pregunta 1: ¿Cuál es la diferencia entre el NameNode y el DataNode en HDFS? Respuesta: El NameNode gestiona metadatos en RAM (namespace, inodos, mapeo de bloques). Los DataNodes almacenan y leen físicamente los bloques en disco.
Pregunta 2: ¿Por qué se utilizan los puertos 9870 y 9000 en el NameNode? Respuesta: El puerto 9870 es la Web UI HTTP (Hadoop 3.x) y el 9000 es el puerto IPC/RPC binario para operaciones cliente.
Pregunta 3: ¿Qué ventaja ofrece Hadoop Streaming frente al MapReduce en Java? Respuesta: Usa flujos estándar (stdin/stdout), permitiendo scripts en Python o Bash sin compilar código Java.
Pregunta 4: ¿Cómo garantiza el clúster la tolerancia a fallos y la persistencia de datos? Respuesta: Volúmenes Docker nombrados conservan fsimage y bloques en el host; HDFS redistribuye réplicas si un DataNode cae.

5. Checklist Pre-Presentación y Commits del Repositorio

- **Docker Engine:** Activo y operativo en segundo plano.
- **Contenedores:** 5 servicios en estado Up (docker ps) | Web UI HDFS: http://localhost:9870 | YARN: http://localhost:8088
- **Terminal:** Carpeta del proyecto lista para demostración interactiva en consola Linux.
- **Historial de Commits:** Commits atómicos escalonados de Rjoel <svr0035567@est.univalle.edu>.